Logged in

Select Mode

Preventive Mode

Logout

📊 DQDash - AI Data Quality Assistant

🛡 Preventive Mode - AI Database Design Advisor

Describe your business idea and the system will suggest an optimized database schema.

Enter Business Description

Design a database for an online learning platform like Udemy.

The system should support:
- instructors who create courses
- students who enroll in courses
- courses with multiple lessons
- student progress tracking
- course ratings and reviews
- payments for course purchases

Select LLM Model

phi3

Generate Database Design

📋 AI Generated Database Design

⚠ This AI-generated schema is a draft recommendation. Final validation by a database architect is recommended.

Database Design

SECTION 1: DATABASE DESIGN

Tables and Columns with Data Types

- Courses (courseID INT, courseName VARCHAR(255), instructorID INT, lessonCount INT, rating DECIMAL(3,2), review TEXT) Primary Key = courseID
- Instructors (instructorID INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(255), bio TEXT, email VARCHAR(255)) Unique index on instructorEmail for uniqueness.
- Students (studentID INT AUTO_INCREMENT Primary Key, fullName VARCHAR(255), age INT CHECK(age > 18), gender ENUM('Male', 'Female', 'Other'), email VARCHAR(255) UNIQUE()) Unique index on studentEmail for uniqueness.
- CourseEnrollments (enrollmentID INT AUTO_INCREMENT PRIMARY KEY, courseID INT REFERENCES Courses(courseID), studentID INT REFERENCES Students(studentID), status ENUM('Pending', 'Active', 'Completed')) Foreign Key = enrollmentID
- Payments (paymentID INT AUTO_INCREMENT PRIMARY KEY, courseID INT NOT NULL REFERENCES Courses(courseID), paymentAmount DECIMAL(10,2), transactionDate DATETIME) Unique index on email for uniqueness.
- Lessons (lessonID INT AUTO_INCREMENT Primary Key, lessonNumber INTEGER, courseID INT REFERENCES Courses(courseID)) Foreign Key = lessonID
- StudentProgress (progressID INT PRIMARY KEY, studentID INT NOT NULL REFERENCES Students(studentID), enrollmentID INT NOT NULL REFERENCES CourseEnrollments(enrollmentID), completedLessons TINYINT, totalLessons ENUM('Pending', 'Active'), completionPercentage DECIMAL(5,2)) Foreign Key = progressID

- Reviews (reviewID INT AUTO_INCREMENT PRIMARY KEY, courseID INT REFERENCES Courses(courseID), studentID INT REFERENCES Students(studentID), content TEXT, rating DECIMAL(3,2), reviewDate DATETIME) Foreign Key = reviewID

Normalization Explanation: The schema is designed in the Third Normal Form (3NF). The Courses table has a primary key and does not have transitive dependencies. Instructors are referenced by course instructor ID which relates to their courses through foreign keys ensuring that there cannot be duplicates of data within related tables without proper referencing, maintaining consistency across our database system.

SECTION 2: SQL CREATE TABLE SCRIPT

```sql
CREATE TABLE Courses (
    courseID INT AUTO_INCREMENT PRIMARY KEY,
    courseName VARCHAR(255),
    instructorID INT NOT NULL,
    lessonCount INT,
    rating DECIMAL(3,2),
    review TEXT,
    FOREIGN KEY (instructorID) REFERENCES Instructors(instructorID) ON DELETE SET NULL
);

CREATE TABLE Instructors (
    instructorID INT AUTO0_INCREMENT PRIMARY KEY,
    name VARCHAR(255),
    bio TEXT,
    email VARCHAR(255) UNIQUE CHECK (email REGEXP '^[A-Za-z0-9._%+-]+@[A-Za-z]{2,}$') -- Ensuring an actual e-mail format.
);

CREATE TABLE Students (
    studentID INT AUTO_INCREMENT PRIMARY KEY,
    fullName VARCHAR(255),
    age INT CHECK(age > 18),
    gender ENUM('Male', 'Female', 'Other'),
    email VARCHAR(255) UNIQUE
);

CREATE TABLE CourseEnrollments (
    enrollmentID INT AUTO_INCREMENT PRIMARY KEY,
    courseID INT NOT NULL REFERENCES Courses(courseID),
    studentID INT NOT NULL REFERENCES Students(studentID),
    status ENUM('Pending', 'Active', 'Completed'),
    FOREIGN KEY (enrollmentID) REFERENCES CourseEnrollments(enrollmentID) ON DELETE SET NULL,
    CHECK ((status IN ('Pending', 'Active'))) -- Only allowing valid statuses.
);

CREATE TABLE Payments (
    paymentID INT AUTO_INCREMENT PRIMARY KEY,
    courseID INT NOT NULL REFERENCES Courses(courseID),
    paymentAmount DECIMAL(10,2),
    transactionDate DATETIME UNIQUE CHECK ((transactionDate >= '2007-01-01')) -- Assuming transactions cannot be older than 2 years.
);

CREATE TABLE Lessons (
    lessonID INT AUTO_INCREMENT PRIMARY KEY,
    lessonNumber INTEGER NOT NULL,
    courseID INT REFERENCES Courses(courseID),
    FOREIGN KEY (lessonID) REFERENCES CourseEnrollments(enrollmentID) ON DELETE SET NULL -- Linking Lessons to a specific student and the corresponding enrolled course
```

```
);

CREATE TABLE StudentProgress (
    progressID INT PRIMARY KEY,
    studentID INT NOT NULL REFERENCES Students(studentID),
    enrollmentID INT NOT NULL REFERENCES CourseEnrollments(enrollmentID),
    completedLessons TINYINT CHECK ((completedLessons BETWEEN 0 AND lessonCount)), -- To ensure that the number of complete lessons is not greater than what's expected
    totalLessons ENUM('Pending', 'Active'),
    completionPercentage DECIMAL(5,2) CHECK ((completionPercentage BETWEEN 0 AND 100)) -- To ensure percentage calculation within bounds of [0, 100].
);

CREATE TABLE Reviews (
    reviewID INT AUTO_INCREMENT PRIMARY KEY,
    courseID INT REFERENCES Courses(courseID),
    studentID INT REFERENCES Students(studentID),
    content TEXT NOT NULL,
    rating DECIMAL(3,2) CHECK (rating BETWEEN 1 AND 5), -- Assuming ratings are on a scale of 1 to 5.
    reviewDate DATETIME UNIQUE CHECK ((reviewDate >= '2020-01-01')) -- Ignoring reviews older than the platform's start date for simplicity in this model, which is Jan
);
```

SECTION 3: ER Diagram Description EntityRelationship (Courses) -1--> EntityInstructors --*---> EntityLessons\n / | /| /|\n EntityEnrollments EntityProgress StudentID <-\n (Students, Emails & other details not directly related to courses or the learning process) -1--> +- /---> Reviews --+-(CourseID)\n | | / CourseRatings\n EntityPayments (each payment for a course is unique and belongs only to one purchase of an enrolled student's completion) -1--> +- /---> StudentProgress --+-\n

SQL Script

```
**SECTION 1: DATABASE DESIGN**

_Tables and Columns with Data Types_
- Courses (courseID INT, courseName VARCHAR(255), instructorID INT, lessonCount INT, rating DECIMAL(3,2), review TEXT) Primary Key = courseID
- Instructors (instructorID INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(255), bio TEXT, email VARCHAR(255)) Unique index on instructorEmail for uniqueness.
- Students (studentID INT AUTO_INCREMENT Primary Key, fullName VARCHAR(255), age INT CHECK(age > 18), gender ENUM('Male', 'Female', 'Other'), email VARCHAR(255) UNIQUE
- CourseEnrollments (enrollmentID INT AUTO_INCREMENT PRIMARY KEY, courseID INT REFERENCES Courses(courseID), studentID INT REFERENCES Students(studentID), status ENUM(
- Payments (paymentID INT AUTO_INCREMENT PRIMARY KEY, courseID INT NOT NULL REFERENCES Courses(courseID), paymentAmount DECIMAL(10,2), transactionDate DATETIME) Unique
- Lessons (lessonID INT AUTO_INCREMENT Primary Key, lessonNumber INTEGER, courseID INT REFERENCES Courses(courseID)) Foreign Key = lessonID
- StudentProgress (progressID INT PRIMARY KEY, studentID INT NOT NULL REFERENCES Students(studentID), enrollmentID INT NOT NULL REFERENCES CourseEnrollments(enrollment
- Reviews (reviewID INT AUTO_INCREMENT PRIMARY KEY, courseID INT REFERENCES Courses(courseID), studentID INT REFERENCES Students(studentID), content TEXT, rating DECIM

_Normalization Explanation:_
The schema is designed in the Third Normal Form (3NF). The Courses table has a primary key and does not have transitive dependencies. Instructors are referenced by cou

**SECTION 2: SQL CREATE TABLE SCRIPT**
```sql
CREATE TABLE Courses (
    courseID INT AUTO_INCREMENT PRIMARY KEY,
    courseName VARCHAR(255),
    instructorID INT NOT NULL,
    lessonCount INT,
    rating DECIMAL(3,2),
    review TEXT,
    FOREIGN KEY (instructorID) REFERENCES Instructors(instructorID) ON DELETE SET NULL
);
```
```

```
CREATE TABLE Instructors (
    instructorID INT AUTO0_INCREMENT PRIMARY KEY,
    name VARCHAR(255),
    bio TEXT,
    email VARCHAR(255) UNIQUE CHECK (email REGEXP '^[A-Za-z0-9._%+-]+@[A-Za-z]{2,}$') -- Ensuring an actual e-mail format.
);

CREATE TABLE Students (
    studentID INT AUTO_INCREMENT PRIMARY KEY,
    fullName VARCHAR(255),
    age INT CHECK(age > 18),
    gender ENUM('Male', 'Female', 'Other'),
    email VARCHAR(255) UNIQUE
);

CREATE TABLE CourseEnrollments (
    enrollmentID INT AUTO_INCREMENT PRIMARY KEY,
    courseID INT NOT NULL REFERENCES Courses(courseID),
    studentID INT NOT NULL REFERENCES Students(studentID),
    status ENUM('Pending', 'Active', 'Completed'),
    FOREIGN KEY (enrollmentID) REFERENCES CourseEnrollments(enrollmentID) ON DELETE SET NULL,
    CHECK ((status IN ('Pending', 'Active'))) -- Only allowing valid statuses.
);

CREATE TABLE Payments (
    paymentID INT AUTO_INCREMENT PRIMARY KEY,
    courseID INT NOT NULL REFERENCES Courses(courseID),
    paymentAmount DECIMAL(10,2),
    transactionDate DATETIME UNIQUE CHECK ((transactionDate >= '2007-01-01')) -- Assuming transactions cannot be older than 2 years.
);

CREATE TABLE Lessons (
    lessonID INT AUTO_INCREMENT PRIMARY KEY,
    lessonNumber INTEGER NOT NULL,
    courseID INT REFERENCES Courses(courseID),
    FOREIGN KEY (lessonID) REFERENCES CourseEnrollments(enrollmentID) ON DELETE SET NULL -- Linking Lessons to a specific student and the corresponding enrolled course
);

CREATE TABLE StudentProgress (
    progressID INT PRIMARY KEY,
    studentID INT NOT NULL REFERENCES Students(studentID),
    enrollmentID INT NOT NULL REFERENCES CourseEnrollments(enrollmentID),
    completedLessons TINYINT CHECK ((completedLessons BETWEEN 0 AND lessonCount)), -- To ensure that the number of complete lessons is not greater than what's expected
    totalLessons ENUM('Pending', 'Active'),
    completionPercentage DECIMAL(5,2) CHECK ((completionPercentage BETWEEN 0 AND 100)) -- To ensure percentage calculation within bounds of [0, 100].
);

CREATE TABLE Reviews (
    reviewID INT AUTO_INCREMENT PRIMARY KEY,
    courseID INT REFERENCES Courses(courseID),
    studentID INT REFERENCES Students(studentID),
    content TEXT NOT NULL,
    rating DECIMAL(3,2) CHECK (rating BETWEEN 1 AND 5), -- Assuming ratings are on a scale of 1 to 5.
    reviewDate DATETIME UNIQUE CHECK ((reviewDate >= '2020-01-01')) -- Ignoring reviews older than the platform's start date for simplicity in this model, which is Jan
);
```

```
**SECTION 3: ER Diagram Description**
EntityRelationship (Courses) -1--> EntityInstructors --*---> EntityLessons\n
                    \/              |             /|            /|\n
       EntityEnrollments         EntityProgress   StudentID <-\n
(Students, Emails & other details not directly related to courses or the learning process) -1-->  +-    /---> Reviews --+-(CourseID)\n
                                    |              |            \/      CourseRatings\n
EntityPayments (each payment for a course is unique and belongs only to one purchase of an enrolled student's completion) -1-->  +-    /---> StudentProgress --+-\n
```

ER Diagram (Text)

SECTION 1: DATABASE DESIGN

Tables and Columns with Data Types

- Courses (courseID INT, courseName VARCHAR(255), instructorID INT, lessonCount INT, rating DECIMAL(3,2), review TEXT) Primary Key = courseID
- Instructors (instructorID INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(255), bio TEXT, email VARCHAR(255)) Unique index on instructorEmail for uniqueness.
- Students (studentID INT AUTO_INCREMENT Primary Key, fullName VARCHAR(255), age INT CHECK(age > 18), gender ENUM('Male', 'Female', 'Other'), email VARCHAR(255) UNIQUE()) Unique index on studentEmail for uniqueness.
- CourseEnrollments (enrollmentID INT AUTO_INCREMENT PRIMARY KEY, courseID INT REFERENCES Courses(courseID), studentID INT REFERENCES Students(studentID), status ENUM('Pending', 'Active', 'Completed')) Foreign Key = enrollmentID
- Payments (paymentID INT AUTO_INCREMENT PRIMARY KEY, courseID INT NOT NULL REFERENCES Courses(courseID), paymentAmount DECIMAL(10,2), transactionDate DATETIME) Unique index on email for uniqueness.
- Lessons (lessonID INT AUTO_INCREMENT Primary Key, lessonNumber INTEGER, courseID INT REFERENCES Courses(courseID)) Foreign Key = lessonID
- StudentProgress (progressID INT PRIMARY KEY, studentID INT NOT NULL REFERENCES Students(studentID), enrollmentID INT NOT NULL REFERENCES CourseEnrollments(enrollmentID), completedLessons TINYINT, totalLessons ENUM('Pending', 'Active'), completionPercentage DECIMAL(5,2)) Foreign Key = progressID
- Reviews (reviewID INT AUTO_INCREMENT PRIMARY KEY, courseID INT REFERENCES Courses(courseID), studentID INT REFERENCES Students(studentID), content TEXT, rating DECIMAL(3,2), reviewDate DATETIME) Foreign Key = reviewID

Normalization Explanation: The schema is designed in the Third Normal Form (3NF). The Courses table has a primary key and does not have transitive dependencies. Instructors are referenced by course instructor ID which relates to their courses through foreign keys ensuring that there cannot be duplicates of data within related tables without proper referencing, maintaining consistency across our database system.

SECTION 2: SQL CREATE TABLE SCRIPT

```sql
CREATE TABLE Courses (
    courseID INT AUTO_INCREMENT PRIMARY KEY,
    courseName VARCHAR(255),
    instructorID INT NOT NULL,
    lessonCount INT,
    rating DECIMAL(3,2),
    review TEXT,
    FOREIGN KEY (instructorID) REFERENCES Instructors(instructorID) ON DELETE SET NULL
);

CREATE TABLE Instructors (
    instructorID INT AUTO0_INCREMENT PRIMARY KEY,
    name VARCHAR(255),
    bio TEXT,
    email VARCHAR(255) UNIQUE CHECK (email REGEXP '^[A-Za-z0-9._%+-]+@[A-Za-z]{2,}$') -- Ensuring an actual e-mail format.
);
```

```sql
CREATE TABLE Students (
    studentID INT AUTO_INCREMENT PRIMARY KEY,
    fullName VARCHAR(255),
    age INT CHECK(age > 18),
    gender ENUM('Male', 'Female', 'Other'),
    email VARCHAR(255) UNIQUE
);

CREATE TABLE CourseEnrollments (
    enrollmentID INT AUTO_INCREMENT PRIMARY KEY,
    courseID INT NOT NULL REFERENCES Courses(courseID),
    studentID INT NOT NULL REFERENCES Students(studentID),
    status ENUM('Pending', 'Active', 'Completed'),
    FOREIGN KEY (enrollmentID) REFERENCES CourseEnrollments(enrollmentID) ON DELETE SET NULL,
    CHECK ((status IN ('Pending', 'Active'))) -- Only allowing valid statuses.
);

CREATE TABLE Payments (
    paymentID INT AUTO_INCREMENT PRIMARY KEY,
    courseID INT NOT NULL REFERENCES Courses(courseID),
    paymentAmount DECIMAL(10,2),
    transactionDate DATETIME UNIQUE CHECK ((transactionDate >= '2007-01-01')) -- Assuming transactions cannot be older than 2 years.
);

CREATE TABLE Lessons (
    lessonID INT AUTO_INCREMENT PRIMARY KEY,
    lessonNumber INTEGER NOT NULL,
    courseID INT REFERENCES Courses(courseID),
    FOREIGN KEY (lessonID) REFERENCES CourseEnrollments(enrollmentID) ON DELETE SET NULL -- Linking Lessons to a specific student and the corresponding enrolled course
);

CREATE TABLE StudentProgress (
    progressID INT PRIMARY KEY,
    studentID INT NOT NULL REFERENCES Students(studentID),
    enrollmentID INT NOT NULL REFERENCES CourseEnrollments(enrollmentID),
    completedLessons TINYINT CHECK ((completedLessons BETWEEN 0 AND lessonCount)), -- To ensure that the number of complete lessons is not greater than what's expected
    totalLessons ENUM('Pending', 'Active'),
    completionPercentage DECIMAL(5,2) CHECK ((completionPercentage BETWEEN 0 AND 100)) -- To ensure percentage calculation within bounds of [0, 100].
);

CREATE TABLE Reviews (
    reviewID INT AUTO_INCREMENT PRIMARY KEY,
    courseID INT REFERENCES Courses(courseID),
    studentID INT REFERENCES Students(studentID),
    content TEXT NOT NULL,
    rating DECIMAL(3,2) CHECK (rating BETWEEN 1 AND 5), -- Assuming ratings are on a scale of 1 to 5.
    reviewDate DATETIME UNIQUE CHECK ((reviewDate >= '2020-01-01')) -- Ignoring reviews older than the platform's start date for simplicity in this model, which is Jan
);
```

SECTION 3: ER Diagram Description EntityRelationship (Courses) -1--> EntityInstructors --*---> EntityLessons\n / | /| /|\n EntityEnrollments EntityProgress StudentID <-\n (Students, Emails & other details not directly related to courses or the learning process) -1--> +- /---> Reviews --+-(CourseID)\n | | / CourseRatings\n EntityPayments (each payment for a course is unique and belongs only to one purchase of an enrolled student's completion) -1--> +- /---> StudentProgress --+-\n

⬇ Download SQL Schema

⬇ Download ER Diagram Text

⬇ Download Design Report

⬇ Download Full Architecture Report